

Pict'Express — Project Report

Elaboration Phase — Unified Process

Collaboration: LBA Marseille • ESIEE Paris

Supervisors: Adrien UGON, Akram REDJDAL

Table of Contents

1. [Introduction](#)
 2. [System Architecture](#)
 3. [Design Choices and Patterns](#)
 4. [OWL Ontology](#)
 5. [Unified Process Phase Status](#)
 6. [Weekly Timeline](#)
 7. [Work Distribution — Actual Team \(3 people\)](#)
 8. [Work Distribution — Ideal Team \(6 people\)](#)
 9. [Known Limitations and Future Roadmap](#)
-

1. Introduction

Pict'Express is a mobile application that supports communication between teachers and students with autism spectrum disorder or language and speech difficulties. It converts spoken French instructions into sequences of pictograms, and provides students with a communication board to express personal needs.

The application is deliberately scoped to a single domain — **elementary school, with a focus on mathematics and daily routine** — to keep the vocabulary manageable and the ontology verifiable during the Elaboration phase.

Target users are teachers, support teachers (AESH), and students. Parents and speech therapists are secondary users foreseen for later phases.

Two distinct flows are implemented.

Flow 1 — Teacher to students. The teacher speaks a French instruction. The system transcribes it, extracts the relevant concepts, reasons over an OWL ontology using SWRL rules, and returns a set of pictograms representing the action, the mathematical concept involved, and the required school materials.

Flow 2 — Student to teacher. The student opens a communication board, browses a categorised grid of need pictograms (hunger, fatigue, confusion, ...), taps one, and shows the enlarged image to the teacher. No speech processing or server is required for this flow.

Methodology: Unified Process (UP), iterative and architecture-centric. Team of three, approximately one working day per week over twelve weeks.

2. System Architecture

The system is organised in four layers.

Presentation — a Flutter mobile application with three screens. The Home screen lets the user choose their role. The Teacher screen handles audio recording and displays the returned pictograms. The Child screen shows the communication board and responds to pictogram selection.

Application — a FastAPI server running on Python. It exposes an audio transcription endpoint and a needs query endpoint, and serves pictogram image files as static resources.

Domain — the `OntologyService` class, which encapsulates all ontology logic: loading the OWL file, indexing French labels and ARASAAC identifiers, running the SWRL reasoner at inference time, and grouping needs for the communication board.

Infrastructure — the external components the domain layer depends on: the Whisper speech-to-text model, the spaCy French NLP model, the Pellet OWL reasoner (via `owlready2` and a Java runtime), and the ARASAAC CDN for image delivery.

The 7-step pipeline for Flow 1 is fully implemented:

1. Voice input captured by the Flutter `record` plugin
2. Audio transcribed to French text by Whisper
3. Text lemmatised and filtered by spaCy
4. Lemmas mapped to OWL classes via `OntologyService.find_class_by_label()`
5. SWRL rules executed by Pellet to infer required tools
6. Pictogram identifiers resolved via `OntologyService.get_arasaac_id()`

7. Images displayed in the Flutter UI via network requests

Flow 2 uses a simpler path: the student taps a pictogram from a built-in list; the image loads directly from the ARASAAC public CDN with no server involvement.

3. Design Choices and Patterns

Service Layer

The ontology logic lives entirely in `OntologyService`, a class separate from the FastAPI routing layer. The server calls `infer_pictograms()` or `get_needs_grouped()` without knowing anything about owlready2 or Pellet. This makes the domain testable independently of the HTTP layer — all 30 automated tests run without starting the server.

The alternative — keeping logic inline in the route handler — was the initial state of the project (a hardcoded six-word dictionary in `interroger_ontologie()`). It was replaced precisely because it made testing and extension impossible.

Lifespan Singleton

Loading Whisper, spaCy, and the ontology (which starts a JVM for Pellet) takes 30–60 seconds and must happen once, not per request. FastAPI's lifespan context manager handles this: all models are loaded at startup and attached to `app.state`. Requests access pre-loaded objects.

Async Adapter for CPU-Bound Work

`whisper.transcribe()` is a blocking call. In an async FastAPI endpoint it would stall the event loop. It is wrapped with `asyncio.to_thread()` to run in a thread pool while the event loop remains free for other requests.

SWRL Rule Parser (Strategy workaround)

owlready2 0.50 silently ignores `DLSafeRule` elements when loading OWL/XML files — it uses an older RDF vocabulary internally. Rather than abandoning the OWL file as the rule source, `OntologyService._parse_swrl_rules()` reads the `DLSafeRule` elements from the raw XML using the standard library's `ElementTree`, converts each atom to owlready2's string rule format, and registers the rules programmatically at load time. The OWL file remains the canonical source; Python only interprets it at startup.

Repository (implicit)

`OntologyService` is the single gateway to all ontology data. No other module imports `owlready2` directly. This insulates the rest of the codebase from the OWL/reasoner stack and makes it straightforward to replace the reasoner in the future.

Static Resource Mount

Pictogram PNG files in the `pictogrammes/` directory are served by FastAPI's `StaticFiles` mount. This decouples file management from request routing and allows the Flutter app to load images with a simple URL.

Navigator Pattern (Flutter)

Navigation between the three screens uses `Navigator.push()` with `MaterialPageRoute`. No named routes or routing library were introduced — the three-screen structure does not warrant that complexity.

GRASP Principles Applied

Information Expert. `OntologyService` owns all ontology knowledge and is the only class that queries it.

Low Coupling. The Flutter presentation layer has no knowledge of `owlready2`, `Pellet`, or `spaCy`. It communicates only over HTTP.

High Cohesion. Each class has a single, well-defined responsibility — routing, ontology access, or UI rendering.

Creator. `OntologyService` creates temporary OWL individuals during inference and destroys them via `destroy_entity()` after collecting the results.

GoF Patterns Absent by Choice

Observer, Bloc, and other state management patterns were not introduced. `setState()` is sufficient for a two-flow prototype with no shared cross-screen state. Factory and Strategy patterns were not needed: object creation is trivial and there is a single reasoner backend. These are candidates for the Construction phase if the architecture grows.

4. OWL Ontology

File: `maths.owl` — OWL 2 XML syntax.

IRI: `http://www.pictexpress.org/ontologies/math.owl`

The ontology contains 62 classes, 10 object properties, 34 French label annotations, 34 ARASAAC integer identifiers, 17 SWRL DLSafeRule axioms, and 7 named tool singleton individuals.

Class Hierarchy

The six top-level categories are declared mutually disjoint:

- **Actions** — MathAction (Draw, Calculate, Count, Measure) and ManualAction (Write, Erase, Cut)
- **MathematicalConcept** — GeometricShape (Circle, Square, Triangle, Rectangle, Pentagon, Hexagon), MathOperation (Addition, Subtraction, Multiplication, Division), Magnitude, Number
- **SchoolMaterial** — BasicTool (Pencil, Eraser, Notebook), GeometryTool (Compass, Ruler, SetSquare), CalculationTool (Calculator)
- **Needs** — PhysicalNeed (Hungry, Thirsty, NeedToilet, Hurt), MentalNeed (Tired, NeedBreak, TooMuchNoise), HelpNeed (DoNotUnderstand), ValidationNeed (IFinish, IUnderstand, IsItGood)
- **Person** — Student, Teacher, Assistant
- **CalculationMethod** — Hands, Mind

SWRL Rules

Seventeen DLSafeRule axioms link actions and concepts to required tools. Rules are DL-Safe, meaning they fire only on named individuals. Seven tool singleton individuals

(`theCompass` , `thePencil` , `theRuler` , `theSetSquare` , `theCalculator` , `theNotebook` , `theEraser`) are declared in the ontology to satisfy this requirement.

The rules cover the following patterns:

- Draw + Circle requires Compass and Pencil
- Draw + Square requires Ruler and Pencil
- Draw + Triangle requires SetSquare and Pencil
- Draw + Rectangle requires Ruler and Pencil
- Draw + Pentagon requires Ruler and Pencil
- Draw + Hexagon requires Ruler and Pencil

- Measure + any GeometricShape requires Ruler
- Calculate requires Calculator
- Write requires Pencil and Notebook
- Erase requires Eraser

Key Modelling Decisions

SWRL over OWL restrictions. OWL existential restrictions would infer tools whenever a geometric concept appears, regardless of context. SWRL rules require both the action verb and the concept to be present — "Tracez un cercle" yields tools; "le cercle est rond" does not. This matches the use case more faithfully.

PhysicalNeed as a separate branch. Hunger, thirst, pain, and toilet needs are bodily states, categorically distinct from cognitive MentalNeed (fatigue, noise overload). A dedicated branch makes the UI grouping directly derivable from the ontology structure.

CalculationMethod for Hands and Mind. These represent means of calculation, not physical school supplies. Placing them under SchoolMaterial would create a logical inconsistency since SchoolMaterial is disjoint from Person.

Need labels as full phrases. Labels such as "j'ai faim" are display strings for the teacher, not NLP lemmas. The student communication board does not go through the NLP pipeline.

Technical Issues Resolved

Pellet JAR incompatibility. owlready2 0.50 bundles Jena JARs compiled for Java 25. Java 21 supports only up to class version 65. Fix: three `LangRDFXML` classes are replaced with the originals from the official Jena release via `scripts/patch_owlready2_jars.py`.

DLSafeRule not parsed by owlready2. owlready2 uses an older SWRL RDF vocabulary and ignores `DLSafeRule` elements in OWL/XML. Fix: `_parse_swrl_rules()` reads the rules from the raw XML and registers them programmatically.

Windows `file:///` URI bug. owlready2 produces an invalid path on Windows when loading from a `file:///` URI. Fix: the OWL file is copied to a temporary directory under the name matching the ontology IRI filename, and owlready2's `onto_path` mechanism is used for resolution.

5. Unified Process Phase Status

Inception — Completed

The Inception phase established the project vision, restricted the scope to the elementary school domain, selected the technology stack, and produced the initial Use Case diagram and domain model.

Elaboration — Closing

The Elaboration phase mitigated the primary technical risk — integrating an OWL reasoner into a mobile application backend — and produced a functioning end-to-end prototype for both flows. This report, along with the five UML diagrams, closes the phase.

Deliverables produced during Elaboration:

- Use Case Diagram (updated to implemented use cases)
- Class Diagram (four-layer architecture)
- Sequence Diagram for Flow 1 (teacher)
- Sequence Diagram for Flow 2 (student)
- Domain Model
- OWL ontology with 62 classes and 17 SWRL rules
- Functioning prototype with two complete end-to-end flows
- 30 automated tests, all passing

Construction — Not Yet Started

Construction will extend the vocabulary, introduce AI-generated pictograms as a fallback, add Flutter state management, and produce proper widget and integration tests.

Transition — Planned

Transition will deliver an Android/iOS beta on tablet hardware, user documentation, a demo video, and user testing sessions with children under LBA supervision.

6. Weekly Timeline

Approximately twelve working weeks at one day per week per team member, from early February to late April 2026.

Week 1 — Early February. Project kickoff, requirements analysis with supervisors, study of the CAA domain and ARASAAC. Output: initial scope definition and technology choices.

Week 2. Literature review on OWL ontologies and AAC systems, tooling setup (PlantUML, Protégé, Flutter, Python environment). Output: bibliography, tool configuration.

Week 3. Repository setup, first Use Case Diagram, initial domain glossary. Output: Git repository, use case model, partial glossary.

Week 4. First OWL T-Box skeleton in Protégé, Flutter project scaffold. Output: initial ontology file, app navigation skeleton.

Week 5. Whisper integration, offline Python CLI prototype for speech transcription. Output: `prototype_ecole.py` with microphone capture and Whisper transcription.

Week 6. First FastAPI server with a hardcoded pictogram dictionary. Output: `serveur_api.py v1`, server/Flutter HTTP communication verified.

Week 7. Flutter audio recording UI, pictogram display from server. Output: working prototype demonstrating the full Flow 1 pipeline with a fixed vocabulary.

Week 8 — Mid March. Architecture review, identification of the OWL passivity problem (no labels, no arasaacIds, no SWRL). Output: refactoring plan.

Week 9. OWL structural bug fixes, French label annotations, ARASAAC IDs populated via API for 23 classes. Output: enriched `maths.owl` , `analysis/arasaac_mapping.md` .

Week 10. owlready2 + Pellet integration, 11 SWRL rules, `OntologyService` replacing the hardcoded dictionary, 17 automated tests. Output: `feature/owl-integration` branch.

Week 11 — Early April. Needs cluster populated (11 classes), `ChildScreen` communication board, dual-flow navigation, 30 automated tests. Output: `feature/child-communication` branch.

Week 12 — Late April. UML diagrams updated, `PROJECT_REPORT.md` , Elaboration phase closure. Output: `docs/project-report` branch and pull request.

7. Work Distribution — Actual Team (3 people)

The team of three worked approximately one day per week each over twelve weeks, totalling roughly 36 person-days of effort. Scope decisions were essential at this capacity.

Area 1 — Software Engineering and Architecture. Responsibilities: UML design, Unified Process methodology, repository management, cross-module integration, design pattern

selection, code review, technical documentation. Key outputs include all five UML diagrams, the architecture section of the README, the choice of Service Layer and Lifespan Singleton patterns, and this report.

Area 2 — Backend and Intelligence. Responsibilities: FastAPI server implementation, Whisper and spaCy integration, `OntologyService` development, Pellet debugging, ARASAAC CDN integration, pictogram caching, async handling. Key outputs include `serveur_api.py`, `ontology_service.py`, `requirements.txt`, the Pellet JAR patch script, and the automated test suites.

Area 3 — Domain Modelling and Ontology. Responsibilities: OWL ontology design in Protégé, class hierarchy, SWRL rule authoring, ARASAAC identifier mapping, `maths.owl` maintenance, ontology integration documentation. Key outputs include the complete `maths.owl` file with 62 classes and 17 rules, `docs/ONTOLOGY_INTEGRATION.md`, and the ARASAAC mapping analysis.

With 36 person-days of capacity, the Elaboration phase deliberately excluded AI pictogram generation, full NLP disambiguation, and Flutter testing, all deferred to Construction.

8. Work Distribution — Ideal Team (6 people)

A team of six at one day per week over twelve weeks would total roughly 72 person-days — double the actual capacity.

Role 1 — Project Manager / Scrum Master

Responsibilities: stakeholder communication with LBA and ESIEE, backlog management, sprint facilitation, risk tracking, escalation of technical blockers.

Skills required: Agile and UP methodology, planning tools, coordination.

This role would free the technical team from coordination overhead and ensure that supervisors receive regular progress updates without depending on individual developers.

Role 2 — Software Architect / UML Designer

Responsibilities: architectural decisions, the complete UML diagram suite (Use Case, Class, Sequence, Domain Model), pattern governance (GRASP, GoF), code review for architectural consistency.

Skills required: UP methodology, UML, design patterns, separation of concerns.

This role produces all five diagrams in this report and ensures that implementation decisions remain aligned with the architectural model.

Role 3 — Ontology Engineer

Responsibilities: OWL 2 ontology design in Protégé, SWRL rule authoring, ARASAAC identifier mapping, reasoner validation with Hermit and Pellet, domain glossary maintenance.

Skills required: OWL 2, SWRL, Protégé, owlready2, knowledge engineering.

This role owns `maths.owl`, the SWRL DLSafeRule axioms, and the vocabulary extension process.

Role 4 — Backend / NLP Engineer

Responsibilities: FastAPI server, Whisper and spaCy integration, `OntologyService` implementation, async handling, REST API design, ARASAAC download logic, Pellet troubleshooting.

Skills required: Python, FastAPI, asyncio, NLP, REST API design.

This role covers all Python backend files and is responsible for the SWRL rule parsing bridge between the OWL file and owlready2.

Role 5 — Mobile Developer (Flutter)

Responsibilities: UI and UX implementation, navigation, state management, microphone permissions, HTTP client, widget and integration tests.

Skills required: Flutter, Dart, Material Design, Riverpod or Bloc, platform channels.

This role owns all Flutter screens and would introduce proper state management (currently absent) and automated widget tests from the Elaboration phase.

Role 6 — AI / Generative and QA Engineer

Responsibilities: AI pictogram generation as a fallback when ARASAAC does not cover a concept; QA pipeline including unit tests, integration tests, and CI/CD configuration.

Skills required: generative AI APIs (image generation), prompt engineering, pytest, Flutter testing, GitHub Actions.

This role would make AI fallback generation available from Elaboration rather than deferring it to Construction, and would ensure that every pull request is validated by automated tests.

With 72 person-days, the team would have been able to extend the vocabulary to around 100 concepts during Elaboration, include AI generation, add Flutter widget tests, and begin user validation sessions with real students before the end of the phase.

9. Known Limitations and Future Roadmap

Limitations of the Current Prototype

The communication board (Flow 2) stores its pictogram list as constants in the Flutter code. When the ontology is updated, these constants must be updated manually — there is no automatic synchronisation.

The `GET /needs` endpoint and `OntologyService.get_needs_grouped()` method exist in the backend but are not called by the current Flutter app. The child screen was refactored to use direct CDN loading instead, making the server unnecessary for Flow 2.

The HTTP call to the backend is written inline in `TeacherScreen`. There is no dedicated service class for API communication on the Flutter side — this is an extraction that belongs in the Construction phase.

Flutter uses `setState()` for all state management. There is no shared state between screens, no history of past sessions, and no cross-screen reactivity.

No Flutter widget or integration tests exist. UI regressions are only caught by manual testing.

The vocabulary currently covers roughly 34 annotated concepts. Common school verbs such as read, listen, and colour are not yet modelled.

Logging uses `print()` throughout. There is no structured log output suitable for production debugging.

Limitations of the Model

The class diagram shows `ApiService` as a planned abstraction in the presentation layer. This class does not currently exist — HTTP calls are inline in `TeacherScreen`. It is documented as a planned refactoring.

The sequence diagram for Flow 2 reflects the current implementation: pictogram images are loaded from the ARASAAC CDN directly, and the need items are built from hardcoded constants with no backend call.

Construction Phase Priorities

High priority: AI pictogram generation as a fallback for concepts not covered by ARASAAC; Flutter state management with Riverpod; extraction of the API call from TeacherScreen into a dedicated service; Flutter widget and integration tests.

Medium priority: vocabulary extension (read, listen, colour, fraction, equation); live communication board fed by the `/needs` API instead of hardcoded constants; text-to-speech for the student board; structured logging; GitHub Actions CI.

Low priority: MIME validation for audio uploads; offline mode with local pictogram cache; accessibility improvements; internationalisation.

Transition Phase

Transition will deliver an Android and iOS beta on tablet hardware, user documentation for teachers and speech therapists, a demonstration video, and user testing sessions with children under LBA supervision.

UML Diagrams

All `.puml` files are in `docs/uml/`. To render them as PNG images, run:

```
plantuml docs/uml/*.puml
```

- Use Case Diagram: `docs/uml/use_case_diagram.puml`
- Class Diagram: `docs/uml/class_diagram.puml`
- Sequence Diagram — Teacher Flow: `docs/uml/sequence_teacher_flow.puml`
- Sequence Diagram — Child Flow: `docs/uml/sequence_child_flow.puml`
- Domain Model: `docs/uml/domain_model.puml`

```
function hello() {  
  console.log("Hello, World!");  
}
```

This is a blockquote example
It can contain multiple lines of content

List Example

1. First item
 2. Second item
 3. Third item
- Unordered list item
 - Another item
 - Last item

Start using it! Upload your Markdown file or paste content directly.